Sous quelles conditions un agent LLM dépasse-t-il une automatisation à règles ?

Proposition de recentrage du programme agent contre automatisation de LANCE

Projet LANCE

Dernière révision : 2026-10-12 21:25 (Europe/Brussels, UTC+02:00)
Statut : protocole proposé — aucun résultat expérimental nouveau

Résumé

La comparaison actuelle oppose une automatisation conditionnelle sans modèle de langage (D1, politique `rules`) à un agent adaptatif avec modèle (A1, politique `llm`) sur les phases d'analyse et de vérification, à outils, observations initiales et contrat de preuve communs. Aucun gain du modèle n'est encore démontré en laboratoire. Ce document propose de remplacer la question globale « le modèle bat-il les règles ? » par une question conditionnelle et falsifiable : sous quelles conditions mesurables l'écart A1−D1 devient-il positif, à budget et preuves identiques ? Il définit six conditions candidates (nouveauté, variation de surface, bruit et leurres, profondeur de chaînage, failles logiques, protocole inconnu), quatre hypothèses avec critères de réfutation, un protocole apparié avec contrôles négatifs et familles tenues à l'écart, ainsi que des critères d'arrêt et de poursuite. La contribution revendiquée n'est pas une supériorité générale du modèle, mais une carte des conditions d'écart à budget égal, y compris si elle est vide.

Table des matières

1 Objectif et périmètre **2**
1.1 Question reframée 2
1.2 Ce que ce document n'est pas 2

2 Méthode de la présente étude **2**

3 Faits sourcés et point de départ **3**
3.1 Littérature primaire (lue, avec limites) 3
3.2 Point de départ dans LANCE (faits de code vérifiés) 3

4 Taxonomie des conditions d'écart attendu **4**

5 Hypothèses falsifiables **5**

6 Protocole apparié proposé **6**
6.1 Bras, égalisation et gel 6
6.2 Corpus, puissance et anti-contamination 6
6.3 Mesures 7
6.4 Contrôles négatifs et invalidation de lot 7

7 Critères d'arrêt, de poursuite et résultats publiables **7**

8 Intégration dans LANCE : points d'appui et travail requis **7**

9 Discussion et limites **8**

10 Conclusion et prochaines étapes **8**

1 Objectif et périmètre

1.1 Question reframée

Question proposée. À outils, inventaire public, contrat de preuve (strict-v3), scénario et plafonds de ressources identiques, sous quelles conditions pré-enregistrées la politique A1 obtient-elle un écart positif et reproductible sur la politique D1, mesuré d'abord en vrais positifs confirmés par les preuves, et à quel coût par vrai positif confirmé ?

Définitions à première occurrence. Le contrat `strict-v3` est l'évaluateur commun du projet : un constat ne compte comme vrai positif confirmé (TP confirmé) que si les traces d'exécution archivées satisfont les contrôles communs de cible, d'endpoint et de contenu ; une étiquette « confirmé » dans un rapport ne suffit pas, et une détection sans preuve vaut non-confirmation (voir `docs/benchmark/agent-vs-automation-cli.md`). D1 désigne la politique `rules` (automatisation conditionnelle sans modèle) et A1 la politique `llm` (agent adaptatif avec modèle), toutes deux définies dans ce dossier.

Cette formulation corrige trois faiblesses de la question globale. Premièrement, elle sépare la *découverte* d'une faille de son *exploitation* et de sa *vérification* : la littérature montre que ces capacités divergent fortement (section 3). Deuxièmement, elle impose l'égalisation du périmètre, du temps, de la charge de sondage et du coût, sans laquelle un écart peut refléter un budget plutôt qu'une politique. Troisièmement, elle ne présuppose ni nouveauté IoT ni supériorité du modèle : l'écart nul partout est un résultat admissible et publiable comme borne.

Le périmètre immédiat reste les phases 3 (analyse) et 4 (vérification), seules couvertes par le comparateur D1/A1 existant. L'extension à la reconnaissance (phase 2) et à l'intrusion (phase 5), où la planification adaptative pourrait davantage compter, est une question ultérieure explicite, non une hypothèse acquise.

1.2 Ce que ce document n'est pas

Ce texte est un **protocole proposé**. Il ne rapporte aucun run de laboratoire, aucun score nouveau et aucune validation expérimentale. Les faits de code cités proviennent d'une lecture du dépôt à la date de révision ; les faits externes proviennent de sources primaires listées en références, avec leurs limites de transposition. Les mécanismes avancés pour expliquer un écart éventuel sont des **hypothèses**, distinguées comme telles.

2 Méthode de la présente étude

La proposition a été construite en trois temps. (1) Une revue ciblée de sources primaires sur l'évaluation des agents de test d'intrusion et les limites des scanners, recherchées le 2026-10-12 ; seules les sources effectivement ouvertes sont citées comme lues, les autres sont marquées non relues. (2) Un audit du dépôt LANCE portant sur les modes d'exécution (`src/agent/run_modes.py`), l'audit par règles (`src/agent/rules_audit.py`), le comparateur de politiques (`src/benchmark/compare_policies.py`), les altérations de scénarios (`src/benchmark/scenario_alterations.py`) et le protocole existant de ce dossier (`protocol.md`). (3) Deux contre-revues indépendantes, l'une méthodologique (contamination, budgets, puissance, oracle), l'autre orientée comité de programme (nouveauté, contribution, risque de résultat trivial) ; leurs objections retenues sont intégrées au corps du texte, notamment aux sections 5 et 9, et non reléguées en annexe.

Limite de méthode à déclarer : l'audit de code multi-agents n'a pas produit de vérification exhaustive signée ; les affirmations d'implémentation ci-dessous ont été revérifiées par lecture directe des fichiers cités, mais aucune nouveauté d'implémentation de LANCE n'est revendiquée sur cette base.

3 Faits sourcés et point de départ

3.1 Littérature primaire (lue, avec limites)

Exploitation guidée contre découverte. Fang et al. (arXiv :2404.08144v2, 17 avril 2024, article lu) évaluent 15 vulnérabilités *one-day* réelles : avec la description CVE fournie, leur agent GPT-4 exploite 87 % de ces 15 vulnérabilités en *pass@5* [1], contre 0 % pour GPT-3.5, huit modèles ouverts, ZAP et Metasploit ; sans description, le taux tombe à 7 %. Lecture retenue : la valeur démontrée est le chaînage d'exploitation en plusieurs étapes à partir d'une spécification, pas la découverte autonome. Limites : $n = 15$, tâches web et Python uniquement, *pass@5* optimiste, et comparaison asymétrique — un scanner détecte sans exploiter, ce qui ne le rend pas comparable à un taux d'exploitation.

Autonomie limitée sans humain. Gioacchini et al. [2] (arXiv :2410.03225v2, EMNLP Industry 2025, article lu) proposent AutoPenBench, 33 tâches conteneurisées : l'agent autonome atteint 21 % de réussite, l'agent assisté par un humain 64 %. Lecture retenue : l'autonomie complète est aujourd'hui insuffisante ; mesurer des jalons partiels est plus informatif qu'un succès binaire. Limites : services conteneurisés classiques, sans protocoles IoT (MQTT, CoAP), sans hétérogénéité de micrologiciels ni contrainte de sûreté physique.

Organisation de l'agent. Deng et al. [3] (PentestGPT, USENIX Security 2024, article lu) séparent raisonnement global, génération d'opérations et analyse des sorties pour limiter la perte de contexte ; l'ablation sans module de raisonnement produit la plus forte dégradation rapportée. Lecture retenue : le *scaffold* (architecture d'agent) est un facteur confondant majeur — comparer « LLM contre règles » sans geler le scaffold attribue au modèle ce qui peut revenir à l'orchestration.

Failles logiques et validation. La feuille OWASP *Business Logic Security* [4] (consultée le 2026-10-12) établit que les scanners génériques manquent les failles de logique métier faute de connaître les règles de l'application (propriété par objet et par état, transitions de processus en plusieurs étapes, conditions de course). XBOW [5] (blog du 24 juin 2025 et billet GPT-5 du 15 août 2025, lus) rapporte un testeur autonome classé premier du classement américain HackerOne et l'usage de *validateurs* automatisés confirmant chaque faille avant soumission. Lecture retenue : la distinction « puis-je ? / devrais-je ? » par rôle et par contexte est le mécanisme candidat pour les failles d'autorisation. Limites : source fournisseur, web uniquement, aucun taux indépendant de faux positifs ou de faux négatifs, validateur parfois lui-même un modèle donc non déterministe.

Non relu en primaire à ce stade. Cybench [7] (40 tâches CTF) n'est connu ici que par la critique qu'en fait Gioacchini et al. [2] (§1 : tâches ludifiées, indices intégrés, service unique, sans trafic actif). Cette critique émane d'un concurrent et ne doit pas être citée comme un fait établi avant lecture directe. Le banc CTF de NYU [6] (200 défis CSAW validés) est cité à titre de contexte méthodologique, sans appui normatif.

Inconnu structurant. Aucune source primaire relue ne compare un agent LLM à un scanner déterministe à périmètre, temps et coût égaux sur un parc IoT réel (micrologiciels, protocoles embarqués, contraintes de sûreté). La transposition du web et des conteneurs vers l'IoT est donc une hypothèse à tester, pas un appui.

3.2 Point de départ dans LANCE (faits de code vérifiés)

Le dépôt sépare déjà quatre modes d'exécution (`docs/architecture/run-modes.md`) : `lance` (pipeline six phases avec modèle), `rules` (déploiement puis phases 3–4 déterministes, sans

fournisseur ni modèle), `vanilla` (boucle modèle–outils bornée sur un CIDR explicite, deux sondes Nmap structurées) et `scripted` (procédures versionnées `smoke-v1` hors ligne et `discovery-v1`). Le comparateur expérimental D1/A1 impose le même inventaire public, le profil `full`, les phases [3, 4], un seul travailleur, des budgets obligatoires, et réévalue les deux bras avec le même évaluateur strict-v3 ; il conserve les écarts A1−D1 et refuse de remplacer une valeur indisponible par zéro (`docs/benchmark/agent-vs-automation-cli.md`).

Deux limites sont documentées dans le dépôt lui-même et motivent ce recentrage. D'une part, les règles réutilisées « contiennent des connaissances publiques propres aux simulateurs : chemins, identifiants de test, versions de micrologiciels et rôles API/PKI/OTA/cloud » ; la comparaison actuelle « ne constitue donc pas un test indépendant de généralisation ». D'autre part, le catalogue compte 29 scénarios (S1–S29, répartis `dev-public`/`test-public`) et 22 définitions de packs (`f0`–`f21` plus une chaîne manuelle) : un split par scénario laisse les mêmes familles des deux côtés, donc une fuite par famille reste possible. Enfin, `scenario_alterations.py` déclare des transformations (`rotate_ports`, `service_availability`, `tool_output_profile`, `failure_mode`) dont le protocole existant prévient qu'elles peuvent n'exister qu'en aperçu : une variation non déployée réellement n'est pas une expérience d'adaptation.

La figure 1 résume le principe à conserver : seule la politique de décision change entre les bras.

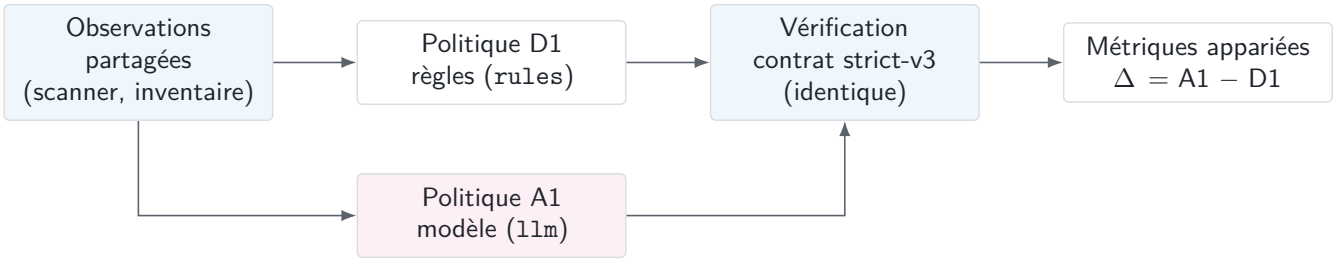


Figure 1 – Principe d'attribution à conserver. Les deux bras partagent observations, outils et contrat de preuve ; seule la politique de décision diffère. Tout écart d'outil, de budget ou de validateur entre bras invalide l'attribution au modèle. Schéma de proposition, pas un résultat.

4 Taxonomie des conditions d'écart attendu

Chaque condition est une variable indépendante à niveaux pré-enregistrés, associée à un mécanisme explicite et à une observation qui le réfuterait. Le tableau 1 les résume ; aucun niveau ne doit être construit en affaiblissant volontairement D1 (règle retirée, parseur bridé) : la condition agit sur le monde audité, pas sur la compétence de la référence.

Condition (niveaux)	**Mécanisme candidat**	**Manipulation / garde-fou**
(a) Nouveauté de famille (en distribution / proche / famille tenue à l'écart)	Généralisation sémantique du modèle contre absence de signature côté règles	Split par famille de packs, jamais par scénario ; famille de test absente des règles, prompts et exemples
(b) Variation de surface (identique / renommages, ports, bannières, ordre permutés)	Invariance sémantique contre fragilité des motifs textuels	Sémantique de faille constante ; *manipulation check* : la variation ne doit pas changer la difficulté intrinsèque, sinon strate distincte
(c) Bruit et leurres (0 / quelques / nombreux services et hôtes leurres)	Tri contextuel du modèle, ou au contraire faux positifs accrus	Leurres à motif connu mais non exploitables ; spécificité mesurée, pas seulement le rappel
(d) Profondeur de chaînage (1 à 4 jalons, 0 à 2 impasses)	Planification et révision multi-étapes (par ex. identifiants par défaut → interface web → MQTT)	Chaînes générées avec branches mortes ; jalons intermédiaires notés, succès final non binaire seul
(e) Failles logiques, d'état et d'autorisation (règle connue / règle absente ou incomplète)	Raisonnement « puis-je / devrais-je » par rôle, objet et état	Politique métier fournie ou non de façon égale aux deux bras ; jamais d'indice privé dans l'entrée publique
(f) Protocole inconnu (connu / documenté non vu / variante propriétaire)	Lecture de documentation et interprétation de sorties ambiguës	Documentation également accessible aux deux bras ; les règles peuvent l'exploiter par extraction, sans signature pré-écrite de la faille

Table 1 – Conditions candidates d'écart A1−D1. Hypothèses de mécanisme à tester ; aucune n'est un résultat. Le contre-domaine où D1 est attendu gagnant (faille à signature connue, contrôle répétable à grande échelle, preuve déterministe exigée) fait partie du plan comme strate de contrôle.

Trois précisions s'imposent. D'abord, la variation de surface (b) n'est un test d'adaptation que si l'information utile reste également lisible par les deux systèmes : une transformation que seul le parseur de D1 ne sait pas lire mesure un écart de parseurs, pas de politiques ; le protocole existant de ce dossier le souligne déjà et nous le reprenons comme critère d'invalidation. Ensuite, le bruit (c) est à double tranchant : l'hypothèse inverse (le modèle, plus crédule face à un leurre plausible, produit davantage de faux positifs) doit être pré-enregistrée au même titre. Enfin, le chaînage (d) ne démontre un pivot que si l'origine, l'accès et la transition réseau sont attestés ; à défaut, seuls les jalons vérifiés sont rapportés, sans revendiquer de pivot causal.

5 Hypothèses falsifiables

Soit Δ l'écart apparié A1−D1 sur le taux de vrais positifs confirmés par strict-v3 (TP confirmés), à scénario, graine de génération et plafonds identiques, et δ un gain minimal utile fixé avant le test sur les données de développement (valeur de travail : 5 points, à recalibrer sur le pilote, jamais après observation du test).

H0 (nulle conditionnelle).
Pour toute condition de la section 4, $\Delta \leq \delta$. Critère de décision unique, pré-enregistré : H0 est rejetée pour une condition si la borne inférieure de l'intervalle de confiance apparié à 95 % de Δ (après correction de Holm) dépasse δ — et non simplement 0. Le même critère vaut pour H1 et pour la poursuite (section 7).

H1 (avantage conditionnel).
Le critère ci-dessus est satisfait uniquement si nouveauté élevée (a, famille tenue à l'écart) ou faille logique sans règle (e) ou protocole inconnu (f) ; il n'est satisfait ni en distribution ni en simple variation de surface. Réfutée dans deux cas symétriques : critère satisfait en distribution (gain générique non prédit, à réexpliquer) ou critère satisfait nulle part (pas d'avantage du modèle).

H2 (résilience différentielle).
Soient $S_{A1}(n)$ et $S_{D1}(n)$ les taux de TP confirmés de chaque bras quand le nombre n de leurres augmente (conditions b et c). H2 prédit que la pente de S_{A1} est moins négative que celle de S_{D1} (A1 se dégrade moins en valeur absolue, par bras). Réfutée si les deux pentes sont égales, ou si la pente de A1 est plus négative, ou si la spécificité de A1 chute avec n (faux positifs en hausse) même à rappel constant.

H3 (coût de l'avantage).
Tout gain de A1 coûte plus de trois fois le coût de D1 par TP confirmé (jetons, appels d'outils et temps mural convertis selon une grille de prix gelée). Réfutée si le ratio est ≤ 3 à gain égal. Un gain réel mais au-delà du ratio borne la viabilité à un usage semi-autonome, sans invalider H1.

Garde-fou métrique commun : un gain en jalons partiels sans gain en TP confirmés vaut inflation de progression, pas avantage ; l'hypothèse correspondante est alors rejetée. Ce garde-fou répond directement à l'enseignement d'AutoPenBench (progression intermédiaire élevée, succès complet faible).

6 Protocole apparié proposé

6.1 Bras, égalisation et gel

Bras principaux D1 et A1 sur les phases 3–4, même inventaire public validé par l'opérateur, mêmes outils et versions, même validateur strict-v3, même état initial de laboratoire restauré entre les bras, ordre randomisé par blocs, au moins trois graines de génération du modèle par configuration (la variance inter-graines est rapportée, jamais moyennée en silence). Bras d'ablation secondaires, sans prétention confirmatoire : modèle sans outil (borne d'interprétation pure) et D1 augmenté d'un oracle de surface (borne de parseur). Avant tout test : gel des règles, prompts, compétences, outils, budgets et du code (empreinte des ressources du run, pas le seul SHA Git), journal d'accès au corpus de test, lecture unique du test.

Les budgets sont égalisés en *plafonds* comparables, pas en unités identiques : temps mural maximal commun incluant les décisions du modèle, charge de sondage commune (appels, cibles, débit — un appel Nmap encapsulant de nombreuses sondes, le seul nombre d'appels ne suffit pas), et plafond monétaire propre à chaque fournisseur converti en coût par TP confirmé selon une grille gelée. Zéro jeton ne signifie pas zéro coût : le calcul et le temps de D1 sont comptés.

6.2 Corpus, puissance et anti-contamination

Le split se fait **par famille de packs** ($f^\star$ jamais présente en développement, dans les règles ni dans les prompts), pas par scénario : des variantes d'un même scénario restent apparentées. Les 29 scénarios existants ne suffisent pas à la puissance ; la génération procédurale de variantes par famille (nouveaux identifiants, secrets, topologies, canaris uniques par instance) fournit l'effectif,

avec une cible explicite : un intervalle de confiance à 95 % de Δ excluant 0 dans la condition testée, dimensionné sur la variabilité du pilote. Chaque variante déployée exige une correspondance vérifiée entre spécification, services réellement déployés, sondes et vérité terrain ; une altération seulement prévue par `scenario_alterations.py` mais non déployée invalide la strate. Analyse : appariement par instance, puis méta-régression $\Delta \sim \text{conditions} + (1|\text{famille})$, correction de Holm sur les tests confirmatoires, résultats par famille publiés avec leurs intervalles.

6.3 Mesures

Primaires : TP confirmés strict-v3 (juge aveugle sur le statut confirmé, binaire), spécificité $\text{TN}/(\text{TN} + \text{FP})$ sur négatifs et leurres, score de jalons (crédit partiel 0–1 par étape analyse $\rightarrow$ vérification $\rightarrow$ preuve $\rightarrow$ accès, grille gelée avant le test). Secondaires : coût par TP confirmé, temps par jalon, taux de rejet de preuve, taux de runs évaluables et causes terminales, variance inter-graines. Les ratios à dénominateur nul restent indéfinis et visibles ; aucune valeur manquante n'est remplacée par zéro, conformément au comparateur existant.

6.4 Contrôles négatifs et invalidation de lot

Quatre contrôles accompagnent chaque campagne : (a) scénario sans faille (spécificité pure) ; (b) preuves tronquées ou invalides, que strict-v3 doit rejeter des deux côtés ; (c) leurre à motif de famille connue mais non exploitable ; (d) canari unique par instance, dont la régurgitation signale une fuite. Un lot est **invalidé**, et non interprété, si un contrôle produit un TP confirmé, si un canari est régurgité, si le *manipulation check* de la condition (b) échoue, ou si les performances en famille tenue à l'écart égalent celles en développement sans explication (soupçon de contamination).

7 Critères d'arrêt, de poursuite et résultats publiables

Arrêt ou invalidation : échec d'un contrôle négatif, canari régurgité, effet qui disparaît après correction de Holm ou qui est dominé par l'effet famille (une seule famille porte l'écart). **Poursuite** (extension aux phases 2 et 5, ou campagne élargie) seulement si le critère de décision de H0/H1 (borne inférieure de l'intervalle à 95 % de Δ supérieure à δ, après Holm) est satisfait en TP confirmés — pas seulement en jalons — dans au moins une condition prédite par H1, coût par TP documenté (H3) et variance inter-graines rapportée.

Tous les résultats suivants sont publiables comme tels, sans reformulation : $\Delta \leq \delta$ partout (réfute l'avantage conditionnel et borne l'autonomie, cohérent avec les 21 % autonomes d'AutoPenBench) ; $\Delta > \delta$ en distribution (réfute la théorie conditionnelle, gain générique à réexpliquer avant toute revendication) ; fragilité de A1 sous bruit (réfute H2) ; gain réel à plus de trois fois le coût par TP (viabilité limitée au semi-autonome). La contribution revendiquée est la **carte des conditions d'écart à budget égal** — y compris vide — et le protocole qui la rend crédible, pas une supériorité générale du modèle ni un nouvel agent.

8 Intégration dans LANCE : points d'appui et travail requis

Points d'appui vérifiés : le sélecteur de modes (`run_modes.py`) isole déjà la politique du reste du pipeline ; le comparateur (`compare_policies.py`) produit les écarts appariés, conserve les scores par bras et exporte les motifs d'incomparabilité ; l'audit applicatif par règles affiche les résultats par équipement sans fabriquer de score. Travail requis, par ordre : (1) un manifeste de campagne versionné portant famille, variante, répétition, ordre, budgets, empreintes d'environnement et causes d'échec (le protocole existant en propose le schéma, aucun nouveau drapeau n'est supposé) ; (2) la génération procédurale de variantes déployables avec vérification spécification–déploiement–vérité terrain, au-delà des aperçus d'altération ; (3) le split par famille et le gel documenté (règles, prompts,

corpus canaris) ; (4) les jalons horodatés et le coût par TP dans la comparaison, en réutilisant le journal d'exécution existant ; (5) seulement ensuite, la décision d'extension aux phases 2 et 5 selon les critères de la section 7. Le pipeline six phases « produit » est gelé comme infrastructure pendant ce programme ; il n'est plus l'objet d'étude.

9 Discussion et limites

Limites externes : les preuves relues sont web et conteneurs ; aucun parc IoT réel, aucun protocole embarqué contraint, aucune exigence de sûreté physique n'y est mesuré. XBOW est une source fournisseur sans taux indépendant ; Cybench reste non relu en primaire. Limites internes : puissance limitée sans génération de variantes, multiplicité des six conditions (d'où Holm et pré-enregistrement), confusion possible entre modèle et scaffold (d'où le gel et les ablations), coût du modèle pouvant masquer un faible gain, et risque trivial symétrique — le modèle perd partout (H0 tient, résultat négatif publiable) ou gagne partout (H1 réfutée, mécanisme à réexpliquer). Aucune réduction du travail humain n'est inférée des jetons ou de l'absence d'intervention ; elle exigerait une étude dédiée avec auditeurs. Les expériences restent dans le périmètre d'audit autorisé du projet (laboratoire NATO) ; ce protocole n'autorise par lui-même aucun run.

10 Conclusion et prochaines étapes

La question globale « le modèle bat-il les règles ? » est remplacée par une question conditionnelle, appariée et falsifiable, dont chaque issue — y compris l'absence d'écart — produit une conclusion utilisable. Prochaines étapes, sans exécution implicite : (1) valider ce cadrage et le seuil δ sur le pilote de développement existant (24 exécutions D1/A1 du modèle de campagne de ce dossier, `benchmarks/experiments/policy-comparison/`, repris au `protocol.md` de ce dossier) ; (2) implémenter le manifeste de campagne et le split par famille ; (3) prouver la génération de variantes déployables sur une famille ; (4) geler, puis seulement lancer la campagne confirmatoire. Toute conclusion retenue à l'issue de la campagne mettra à jour la documentation de référence du benchmark avec son périmètre et ses limites, en lien vers la présente étude.

Références

[1] R. Fang, R. Bindu, A. Gupta, Q. Zhan et D. Kang, *LLM Agents can Autonomously Exploit One-day Vulnerabilities*, arXiv :2404.08144v2, 17 avril 2024. Article lu le 2026-10-12 ; $n = 15$, web/Python, *pass@5*.

[2] L. Gioacchini et al., *AutoPenBench : Benchmarking Generative Agents for Penetration Testing*, arXiv :2410.03225v2, EMNLP Industry 2025. Article lu le 2026-10-12 ; 33 tâches conteneurisées.

[3] G. Deng et al., *PentestGPT : Evaluating and Harnessing Large Language Models for Automated Penetration Testing*, USENIX Security 2024. Article lu le 2026-10-12.

[4] OWASP, *Business Logic Security Cheat Sheet*, cheatsheetseries.owasp.org. Consulté le 2026-10-12 ; date de version non affichée sur la page lue.

[5] N. Waisman (XBOW), *Top-1 : How XBOW did it*, xbow.com, 24 juin 2025 ; O. de Moor et A. Ziegler, *XBOW Unleashes GPT-5's Hidden Hacking Power*, 15 août 2025. Lus le 2026-10-12 ; source fournisseur, web uniquement.

[6] M. Shao et al., *NYU CTF Bench : A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security*, arXiv :2406.05590v3, 2024. Connu par son résumé et sa description de méthode ; non utilisé comme appui normatif ici.

[7] A. Zhang et al., *Cybench : A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models*, ICLR 2025. Non relu en primaire à ce stade ; connu uniquement par la critique de [2], à confirmer avant tout usage normatif.